Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Armstrong sayısı (düzeltme)
  - Önceki derste “kübik sayı” olarak verilen problemin doğru adı Armstrong sayısıdır. Kübik sayılar 8, 27, 64, 125 gibi bir tam sayının küpü olan sayılardır.
  - Genel tanım (ekrandan, 00:24:20): n basamaklı bir sayı için `abcde... = aⁿ + bⁿ + cⁿ + dⁿ + eⁿ + ...` ise sayı Armstrong sayısıdır. Burada n basamak sayısıdır.
  - Üç basamaklı sayılar için koşul: `abc = a³ + b³ + c³`.
  - Bu derste problem yalnızca üç basamaklı sayılar için çözülür. Döngüler öğrenildikten sonra her basamak sayısı için genelleştirilecektir.
- Algoritma 3: Verilen sayının Armstrong sayısı olup olmadığını bulan algoritma (ekrandan, 00:24:20)
  - START → çıkış: `'Üç basamaklı sayı giriniz'` → giriş: `sayı`.
  - İşlem kutusu:
    - `a = sayı DIV 100` (yüzler basamağı)
    - `b = (sayı - (100 * a)) DIV 10` (onlar basamağı)
    - `c = sayı MOD 10` (birler basamağı)
    - `abc = a*a*a + b*b*b + c*c*c`
  - Karar: `abc = sayı` ise `'Armstrong sayı'`, değilse `'Armstrong sayı değil'` yazdırılır → STOP.
  - Onlar basamağı alternatif olarak önce `sayı MOD 100`, sonra `DIV 10` ile de bulunabilir. İşlem sayısı açısından fark yoktur, çünkü mod işlemi kendi içinde zaten çarpma ve çıkarma yapar.
- Akış diyagramlarında üs alma
  - Akış diyagramında `a³`, `a^3` veya `a**3` gösterimi kullanılmaz. Kuvvet, çarpma ile yazılır: `a*a*a`.
  - Kuvvet n kez tekrarlanacaksa ileride for/while döngüsü kullanılacaktır. Programlama dillerinde bunun için `pow` gibi fonksiyonlar vardır.
- Algoritma analizi (kağıt üzerinde)
  - Algoritmanın doğruluğu bir tablo ile kontrol edilir. Tüm değişkenler sütun olarak yazılır, farklı girdiler için değerleri adım adım izlenir.
  - Analiz tablosu (ekrandan, 00:24:20):

sayı	a	b	c	abc	Sonuç
123	1	2	3	36	Armstrong sayı değil
153	1	5	3	153	Armstrong sayı

  - Hem koşulu sağlayan hem sağlamayan bir durum (case) test edilmelidir. Aynı türden durumlar için çok sayıda test gerekmez. Daha karmaşık problemlerde 5-8 farklı durum incelemek gerekebilir.
- Algoritma 4: Başlangıç ve bitiş koordinatları verilen doğrusal eğrinin denklemini hesaplayan algoritma (ekrandan, 01:14:20)
  - Eğri doğrusal kabul edilir: `y = ax + b`.
  - Kağıt üzerindeki çözüm:
    - Eğim: `a = m = (y2 - y1) / (x2 - x1)`
    - `b = y - ax` → bir nokta yerine konarak `b = y1 - a * x1`

    - Sonuç: `y = ax + b`
  - İlk akış diyagramı: START → `'Başlangıç ve bitiş koordinatlarını giriniz'` → giriş: `x1, y1, x2, y2` → işlem: `a = (y2 - y1) / (x2 - x1)`, `b = y1 - a * x1` → çıkış: `'Denklem=', a, 'x+', b` → STOP.
  - Sıfıra bölme (division by zero): `x1 = x2` ise doğru y eksenine paraleldir, eğim tanımsızdır ve bölme işlemi hata verir.
  - Düzeltilmiş akış diyagramı: veriler okunduktan hemen sonra, işlemden önce `x2 = x1` kontrolü yapılır.
    - True kolu: `'x=', x2` yazdırılır (denklem `x = x2` biçiminde bir dikey doğrudur).
    - False kolu: a ve b hesaplanır, `a, 'x+', b` yazdırılır.
    - Kollar birleşir → STOP.
  - `y1 = y2` durumunda ek kontrol gerekmez: a = 0 çıkar ve denklem `0x + b` olarak doğru yazdırılır.
  - İki nokta aynıysa tek bir nokta vardır ve doğru tanımlanamaz. Ancak problem "başlangıç ve bitiş koordinatları verilen doğru" dediği için girdilerin doğru olduğu varsayılır.
  - b negatifse çıktı `x+-5` gibi görünür. Bu görsel olarak hoş değildir ama sonucu yanlış yapmaz.
  - Bu çözüm x eksenine paralel, y eksenine paralel ve eksenleri kesen tüm doğrular için çalışır, yani genel bir çözümdür.
- Bilgisayar bilinmeyenli denklem çözemez
  - Bir öğrencinin çözümünde `y = m * (x - x1) + y1` ifadesi bilgisayara hesaplatılmak istenmiştir. x bilinmediği için bilgisayar bu ifadeyi hesaplayamaz.
  - Bilgisayar yalnızca dört işlem, mod alma ve karşılaştırma yapabilir ve bu işleri tekrar ettirebilir. Sembolik matematik yapan web siteleri de arka planda birinin dört işlemle yazdığı fonksiyonları kullanır.
- Bilginin kaynağı
  - Algoritmada bir girdi varsa onu kullanıcı verir. İlerleyen konularda bilgi dosyalardan, ağ üzerinden (web sitesi, API) gelebilir ya da rastgele üretilebilir.
- Ödev problemi: Kenar uzunlukları verilen üçgenin türü
  - Kenar uzunlukları verilen bir üçgenin türünü (eşkenar, ikizkenar, çeşitkenar) ekrana yazdıran algoritmanın akış diyagramı çizilecektir. Dik üçgen kontrolünün eklenmesi de denenebilir.
  - Kenar uzunluklarının geçerli bir üçgen oluşturduğu varsayılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Çözümleri genelleştirin
  - Özel bir problemi çözmek yerine, o problemi de kapsayan daha genel problemleri çözebilen algoritmalar tasarlamak önemlidir. Armstrong sayısı problemi ileride her basamak sayısı için genelleştirilecektir.
- Akış diyagramı kuralları
  - If kutusunun kolları mutlaka T (true) ve F (false) olarak etiketlenmelidir. Etiketsiz if hatalı kabul edilir. True kolu sağda ya da solda olabilir.
  - Mesajlar tek tırnak içinde yazılmalıdır.
  - Akış yukarıdan aşağıya ilerlemelidir. Aşağı inip sonra yukarı çıkan diyagramlar ödev ve sınavlarda istenmez.
  - Bir algoritmada yalnızca bir STOP olmalıdır. Kollar en sonda birleşmelidir.
  - Giriş ve çıkış ayrı kutularda olmalıdır. Mesaj, giriş kutusunun içine yazılmaz.
  - Derste gösterilen notasyon kullanılmalıdır. Başka araçların (ör. altyazıda "FlowShot" diye geçen uygulama, draw.io) farklı sembolleri bilinmeli, ama ders kapsamında kullanılmamalıdır.
- If içinde mümkün olduğunca az işlem yapın
  - Hesaplamalar dışarıda bir işlem kutusunda yapılmalı, karar kutusunda yalnızca karşılaştırma kalmalıdır. Bu, okunabilirliği artırır.
- İşlem önceliğini parantezle gösterin
  - Hoca, işlem önceliği zaten belli olsa bile ifadeleri parantezle açıkça yazmayı tercih eder: `(sayı - (100 * a)) DIV 10`.
- Kullanıcı hataları
  - Sınavda aksi söylenmedikçe kullanıcının doğru girdi verdiği varsayılır. Ödevlerde ve gerçek hayatta ise kullanıcı hataları (örn. sayının gerçekten üç basamaklı olup olma-

dığı) kontrol edilmelidir.
- Daha anlamlı çıktı mesajları
  - `'Armstrong sayı değil'` yerine `sayı, 'Armstrong sayı değildir'` yazdırmak daha iyidir. Örnek çıktı: “123 Armstrong sayı değildir”. Burada hesaplanan `abc` değil, girilen `sayı` yazdırılmalıdır.
- Sıfıra bölmeyi kontrol edin
  - Bölme işleminin paydası sıfır olabiliyorsa işlemden önce kontrol edilmelidir. Aksi hâlde division by zero hatası oluşur.
- Hata yapmaktan çekinmeyin
  - Paylaşılan çözümlerdeki hatalar herkesin öğrenmesi için değerlidir.

Kısa Tekrar Notları

- Armstrong sayısı: basamak sayısı n olan sayının rakamlarının n. kuvvetleri toplamı kendisine eşittir. $153 = 1^3 + 5^3 + 3^3$.
- Basamaklar: yüzler `sayı DIV 100`, onlar `(sayı - 100*a) DIV 10`, birler `sayı MOD 10`.
- Akış diyagramında kuvvet = çarpma (`a*a*a`). `^` kullanılmaz.
- Analiz: değişken tablosu + koşulu sağlayan ve sağlamayan en az birer test.
- Doğru denklemi: `a = (y2-y1)/(x2-x1)`, `b = y1 - a*x1`.
- `x1 = x2` → division by zero. Önce kontrol et, sonuç `x = x2`.
- If kollarında T/F zorunludur. Tek STOP. Akış yukarıdan aşağı.
- Bilgisayar bilinmeyenli ifade hesaplayamaz, yalnızca bilinen değerlerle işlem yapar.

Detaylı Açıklamalar

Ders, önceki derste ödev olarak verilen problemin çözümüyle başlar. Hoca önce bir düzeltme yapar: basamaklarının küpleri toplamı kendisine eşit olan sayılara “kübik sayı” değil, Armstrong sayısı denir. Kübik sayı ise bir tam sayının küpü olan sayıdır. Armstrong sayısının genel tanımında üs, sayının basamak sayısıdır. Üç basamaklı sayılar için üs 3, dört basamaklı sayılar için 4’tür. Bu derste problem üç basamaklı sayılar için çözülür. Döngüler öğretildikten sonra aynı problem her basamak sayısı için genelleştirilecektir. Hoca bu sayede özel çözüm ile genel çözüm arasındaki farkın görüleceğini, bilgisayar biliminde genelleştirilebilir çözümlerin önemli olduğunu vurgular.

Çözümde sayı tek bir değer olarak okunur ve basamakları matematiksel işlemlerle ayrılır. Yüzler basamağı sayının 100’e tam bölümüyle (`DIV 100`), birler basamağı 10’a bölümünden kalanla (`MOD 10`) bulunur. Onlar basamağı için sayıdan yüzler kısmı çıkarılıp sonuç 10’a tam bölünür. Ardından basamakların küpleri, akış diyagramında üs işareti kullanılamadığı için çarpma ile hesaplanır ve toplanır. Toplam sayıya eşitse sayı Armstrong sayısıdır. Hoca, hesaplamaların karar kutusunun dışında yapılmasını ve karar kutusunda yalnızca karşılaştırmanın kalmasını önerir.

Algoritmanın doğruluğu kağıt üzerinde bir analiz tablosuyla gösterilir. Tüm değişkenler sütun olarak yazılır ve farklı girdiler için değerleri izlenir. 123 için basamaklar 1, 2, 3’tür ve küpler toplamı 36 olduğundan sayı Armstrong sayısı değildir. 153 için basamaklar 1, 5, 3’tür ve küpler toplamı 1 + 125 + 27 = 153 olduğundan sayı Armstrong sayısıdır. Böylece algoritmanın iki olası çıktısı da test edilmiş olur. Bir öğrencinin bir yazılım aracıyla çizdiği alternatif çözüm üzerinden, akışın yukarıdan aşağı ilerlemesi gerektiği ve ders kapsamında derste gösterilen notasyonun kullanılması gerektiği konuşulur.

Derste ikinci bir problem çözülür: başlangıç ve bitiş koordinatları verilen doğrusal bir eğrinin denklemini bulmak. Doğrunun denklemi `y = ax + b` biçimindedir. Eğim `a = (y2 - y1) / (x2 - x1)` ile bulunur. Ardından noktalardan biri denklemde yerine konarak `b = y1 - a * x1` hesaplanır. Bilgisayar bilinmeyenli bir ifadeyi hesaplayamayacağı için, `y = m(x - x1) + y1` gibi x’in bilinmediği bir ifade yerine a ve b’nin sayısal değerleri ayrı ayrı bulunur ve sonuç çıktıda `a, 'x+', b` biçiminde birleştirilir.

Bir öğrencinin önerisiyle önemli bir özel durum ele alınır. x1 ve x2 eşitse doğru y eksenine paraleldir, eğim tanımsızdır ve bölme işlemi sıfıra bölme hatası verir. Bu yüzden veriler okunduktan hemen sonra, bölme işleminden önce `x2 = x1` kontrolü yapılır. Eşitse denklem `x = x2` olarak yazdırılır, değilse a ve b hesaplanıp genel denklem yazdırılır. y1 ve y2’nin eşit olduğu

durumda ise a sıfır çıkar ve denklem doğru şekilde `0x + b` olarak yazılır. Böylece algoritma tüm doğrusal durumlar için çalışan genel bir çözüme dönüşür. İki noktanın aynı olması durumu ise problem tanımı gereği girdilerin doğru verildiği varsayıldığı için ayrıca ele alınmaz.

Ders sonunda bir sonraki derse kadar çözülmek üzere kenar uzunlukları verilen bir üçgenin türünü (eşkenar, ikizkenar, çeşitkenar ve istenirse dik üçgen) belirleyen algoritmanın akış diyagramı ödev olarak verilir. Hoca, bir sonraki derslerde for ve while döngülerinin anlatılacağını ve bunlarla birlikte artık birçok problemin çözülebileceğini belirtir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.